

Module 7: SSDLC, Supporting Tools & ShopCo Lab

Introducing ShopCo • OWASP Bingo • SSDLC Mapping • Supply Chain Demo

Day 2 • Deferred from Day 1 Module 5 | BBC Academy / JBI Training

Module Overview

This module has four linked activities. Parts A and B introduce the ShopCo deliberately vulnerable Flask app as a revision vehicle for Day 1 concepts. Parts C and D extend this into the SSDLC and supply chain topics.

Part	Activity	Time
A	ShopCo Installation	5 mins
B	OWASP Bingo — Identify the 8 Flaws	15 mins
C	SSDLC Mapping — When Should Each Flaw Have Been Caught?	15 mins
D	Supply Chain Demo — pip-audit	10 mins

Part A — ShopCo Installation (5 minutes)

Steps	What to Expect						
Step 1 — Open a Terminal Press Ctrl+Alt+T	A terminal window opens. You should see a prompt ending in \$						
Step 2 — Clone the Repository <pre>cd ~ git clone https://github.com/ TimDWilliams-ProteQC-CTO/ shopco-security-lab.git</pre>	Git downloads all three app versions, the requirements files, and all course materials into ~/shopco-security-lab/						
Step 3 — Install Flask <pre>pip3 install flask</pre> Wait for the Successfully installed message before continuing.	Note for Ubuntu 20.04: do not add --break-system-packages — that flag does not exist on this version and will cause an error.						
Step 4 — Start the App <pre>cd ~/shopco-security-lab python3 vulnerable_shop.py</pre>	You should see: * Running on http://127.0.0.1:5000 * Debug mode: on Open Firefox and go to http://localhost:5000						
Step 5 — Verify Login Accounts <table><tr><th>Username</th><th>Password</th></tr><tr><td>admin</td><td>letmein</td></tr><tr><td>alice</td><td>password123</td></tr></table>	Username	Password	admin	letmein	alice	password123	To stop the app at any time, click in the terminal and press Ctrl+C To reset to a clean state (e.g. if the database gets corrupted): <pre>rm -f shop.db python3 vulnerable_shop.py</pre>
Username	Password						
admin	letmein						
alice	password123						

bob	qwerty	A fresh shop.db is created automatically on restart.
-----	--------	---

Part B — OWASP Bingo: Identify the 8 Flaws (15 minutes)

 **Task:** Open **vulnerable_shop_CLEAN.py** in a text editor (**mousepad ~/shopco-security-lab/vulnerable_shop_CLEAN.py**). Read through the source code and annotate each flaw you find. There are exactly **8 deliberate flaws** mapped to OWASP Top 10:2025 categories.

 **Hints:** Two flaws are in the first 6 lines. The **login()** function contains three. The **profile()** function contains two. The **ping()** function contains the most severe single flaw.

#	Location (function / line)	OWASP Top 10:2025 Category	What could an attacker do with this?
1			
2			
3			
4			
5			
6			
7			
8			

Part C — SSDLC Mapping: When Should Each Flaw Have Been Caught? (15 minutes)

 **Task:** For each of the 8 flaws, decide which phase of a Secure Software Development Lifecycle (SSDLC) should have prevented or detected it. Use the phase reference below.

Phase	What happens at this phase
Requirements	Security requirements & threat modelling — OWASP ASVS, STRIDE, data flow diagrams
Design	Secure design review — authentication model, session handling, access control decisions
Development	Secure coding standards, peer review, developer-run unit tests

SAST / Build	Automated static analysis — Bandit, SonarQube, dependency scanning in CI pipeline
Test / DAST	Dynamic testing — ZAP active scan, penetration testing, IDOR probing
Deployment	Configuration review — DEBUG mode, error handling, secrets management
Operations	Monitoring, patching, incident response

#	Flaw Summary	SSDLC Phase(s)	Control or practice that would have caught it
1	Hardcoded secret key ("password")		
2	DEBUG=True in production config		
3	MD5 used for password hashing		
4	Raw f-string SQL (SQL injection)		
5	Session stores username not user_id		
6	No authentication check on /profile		
7	IDOR — any user_id accepted from URL		
8	Command injection via os.popen()		

Discussion Questions

1. Flaws 4 and 8 share a root cause. What is it, and what general principle prevents both?
2. Three of the eight flaws (5, 6, 7) are invisible to Bandit SAST. Why? What does this tell you about the limits of automation?
3. Flaw 2 (DEBUG=True) is a one-line change. At which SSDLC phase is it cheapest to catch?

Part D — Supply Chain Demo: pip-audit (10 minutes)

 **Task:** Run pip-audit against the deliberately outdated **requirements_vulnerable.txt** included in the ShopCo repository. Observe the CVEs found, then compare against **requirements.txt** (the safe pinned version) to see the difference.

Commands	What to Look For
----------	------------------

Install pip-audit (if not already installed):

```
pip3 install pip-audit
```

Scan the vulnerable requirements file:

```
cd ~/shopco-security-lab  
pip-audit -r  
requirements_vulnerable.txt
```

Scan the safe requirements file:

```
pip-audit -r requirements.txt
```

Vulnerable scan:

Multiple CVEs across Flask, Werkzeug, Jinja2, requests, and cryptography. Note CVSS scores — some are HIGH.

Key CVEs to highlight:

CVE-2019-14806 — Werkzeug debug PIN predictable → RCE

CVE-2023-30861 — Flask session persists after logout

CVE-2019-10906 — Jinja2 sandbox escape

Safe scan:

No vulnerabilities found. All packages current and pinned.



Key Lesson

The **--hash** flag in **requirements.txt** pins each package to a specific file checksum. If a package on PyPI is replaced by a malicious version (a supply chain attack), pip will refuse to install it. This is a one-line defence against a whole class of A06:2025 — Software and Hardware Supply Chain Failures.

SAST tools like Bandit scan your *code*. pip-audit scans your *dependencies*. Both are necessary, and both belong in a CI pipeline. Neither replaces the other.

What these four activities have in common

ShopCo lets you see the same vulnerability from three perspectives simultaneously: as source code (Part B), as a lifecycle failure (Part C), and as an automated tool finding (Part D). This is the developer's view that black-box tools like Juice Shop cannot provide.